



LayerEdge Staking Security Assessment

CertiK Assessed on May 20th, 2025





CertiK Assessed on May 20th, 2025

LayerEdge Staking

The security assessment was prepared by CertiK, the leader in Web3.0 security.

Executive Summary

TYPES

DeFi

ECOSYSTEM

Ethereum (ETH)

METHODS

Formal Verification, Static Analysis

LANGUAGE

Solidity

TIMELINE

Delivered on 05/20/2025

KEY COMPONENTS

N/A

CODEBASE

[LayerEdge](#)[View All in Codebase Page](#)

COMMITTS

base: 6c78ebe5e9cc56d76a8e5c2dc17259c26d2dc52a

final: b7457074df7cdf5c5c2cdfdf1a70c7c200d27986

[View All in Codebase Page](#)

Vulnerability Summary



12

Total Findings

8

Resolved

0

Partially Resolved

4

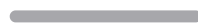
Acknowledged

0

Declined

2 Centralization

2 Acknowledged



Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.

0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

2 Major

2 Resolved



Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.

5 Medium

4 Resolved, 1 Acknowledged



Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

2 Minor

2 Resolved



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

1 Informational

1 Acknowledged



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | LAYEREDGE STAKING

I **Summary**

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

I **Review Notes**

[Overview](#)

[External Dependencies](#)

[Privileged Functions](#)

I **Findings**

[LAF-02 : Contract Upgrade Centralization Risk](#)

[LAF-03 : Centralization Related Risks](#)

[LAF-04 : Incorrect Removal From Fenwick Tree Causing Arithmetic Overflow/Underflow](#)

[LAF-06 : Incorrect Tier Promotion Due to Missing Sufficient Condition Checks in `\\_checkBoundariesAndRecord` function](#)

[LAF-05 : Inefficient Function Call in `\\_checkBoundariesAndRecord`](#)

[LAF-07 : Missing Tier Recalculation in `compoundInterest\(\)` Function for Stakers](#)

[LAF-10 : Unstake May Fail Due To Invalid Tier](#)

[LAS-01 : Incorrect Tier Updates Due to Rank Mismatch](#)

[LAS-02 : Incorrect Rank Assignment Due to Misused Input in Tier Computation](#)

[LAF-08 : Missing Initialization of `ReentrancyGuardUpgradeable` in `LayerEdgeStaking` Contract](#)

[LAF-09 : Check-Effect-Interaction Pattern Violated](#)

[LAF-01 : Inconsistency Between `getCurrentTier` Function Logic and `stakerTierHistory` Usage](#)

I **Appendix**

I **Disclaimer**

CODEBASE | LAYEREDGE STAKING

Repository

LayerEdge

Commit

base: 6c78ebe5e9cc56d76a8e5c2dc17259c26d2dc52a

final: b7457074df7cdf5c5c2cdfdf1a70c7c200d27986

AUDIT SCOPE | LAYEREDGE STAKING

2 files audited ● 1 file with Acknowledged findings ● 1 file with Resolved findings

ID	Repo	File	SHA256 Checksum
● LES	Layer- Edge/edgen- staking	 stake/LayerEdgeStaking.sol	89cc44889bebde7b3485f7bf21f7e750e9d1fef ae9281e63f00a3313c3641645
● FTL	Layer- Edge/edgen- staking	 library/FenwickTree.sol	74d9a40eea4800e56a56c8490cbee6b0c832 7a18adf1d4de6a9e40a3e3125688

APPROACH & METHODS | LAYEREDGE STAKING

This report has been prepared for LayerEdge to discover issues and vulnerabilities in the source code of the LayerEdge Staking project as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Formal Verification and Static Analysis techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

The security assessment resulted in findings that ranged from critical to informational. We recommend addressing these findings to ensure a high level of security standards and industry practices. We suggest recommendations that could better serve the project from the security perspective:

- Testing the smart contracts against both common and uncommon attack vectors;
- Enhance general coding practices for better structures of source codes;
- Add enough unit tests to cover the possible use cases;
- Provide more comments per each function for readability, especially contracts that are verified in public;
- Provide more transparency on privileged activities once the protocol is live.

REVIEW NOTES | LAYEREDGE STAKING

Overview

LayerEdge Staking contracts will be deployed on Ethereum(or other evms) and on LayerEdge's L1 (EVM compatible).

Users can stake `stakingToken` to earn `stakingToken` rewards through the service.

On Ethereum, the staking token will be a simple ERC20 contract(Openzeppelin's implementation) of \$EDGEN token.

On LayerEdge's L1, the staking token will be a WETH9 implementation, a wrapper of native token \$EDGEN.

Each user is classified into tier1, tier2, or tier3 based on specific rules. Each tier corresponds to a different APY. Rewards are calculated based on the user's corresponding APY over a period of time. Users can claim their rewards or choose to stake the rewards to earn new rewards.

External Dependencies

In **LayerEdge Staking**, the system inherits or uses a few of the dependent injection contracts or addresses to fulfill the needs of its business logic. The scope of the audit treats third-party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised, and this may lead to lost or stolen assets.

The following libraries/contracts are considered third-party dependencies:

- **@openzeppelin/contracts/**: This widely used library is utilized to inherit standard implementations of `ERC20` contracts, providing essential features such as token standard compliance.
- **@openzeppelin/contracts-upgradeable**: This widely used library adds more upgradeable features compared to the previous one.
- **stakingToken**: stake and reward token

We assume these contracts or addresses are valid and non-vulnerable actors and implement proper logic to collaborate with the current project.

Privileged Functions

In the **LayerEdge Staking** project, several roles are adopted to operate with some of the key functionalities of the implemented contract. Such privileged roles and methods are described in the findings:

- **LAF-03: Centralization Related Risks**
- **LAF-02: Contract Upgrade Centralization Risk**

The advantage of this privileged role in the codebase is that the client reserves the ability to adjust the protocol according to the runtime required to best serve the community. It is also worth noting the potential drawbacks of these functions, which

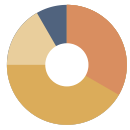
should be clearly stated through the client's action/plan. Additionally, if the private key of the privileged account is compromised, it could have devastating consequences for the project.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community.

Any plan to invoke the aforementioned functions should also be considered to move to the execution queue of the

`TimeLock` contract.

FINDINGS | LAYEREDGE STAKING



12
Total Findings

0
Critical

2
Centralization

2
Major

5
Medium

2
Minor

1
Informational

This report has been prepared to discover issues and vulnerabilities for LayerEdge Staking. Through this audit, we have uncovered 12 issues ranging from different severity levels. Utilizing the techniques of Formal Verification & Static Analysis to complement rigorous manual code reviews, we discovered the following findings:

ID	Title	Category	Severity	Status
LAF-02	Contract Upgrade Centralization Risk	Centralization	Centralization	● Acknowledged
LAF-03	Centralization Related Risks	Centralization	Centralization	● Acknowledged
LAF-04	Incorrect Removal From Fenwick Tree Causing Arithmetic Overflow/Underflow	Logical Issue	Major	● Resolved
LAF-06	Incorrect Tier Promotion Due To Missing Sufficient Condition Checks In <code>_checkBoundariesAndRecord</code> Function	Logical Issue	Major	● Resolved
LAF-05	Inefficient Function Call In <code>_checkBoundariesAndRecord</code>	Logical Issue	Medium	● Resolved
LAF-07	Missing Tier Recalculation In <code>compoundInterest()</code> Function For Stakers	Logical Issue, Inconsistency	Medium	● Resolved
LAF-10	Unstake May Fail Due To Invalid Tier	Denial of Service	Medium	● Resolved
LAS-01	Incorrect Tier Updates Due To Rank Mismatch	Logical Issue	Medium	● Acknowledged
LAS-02	Incorrect Rank Assignment Due To Misused Input In Tier Computation	Logical Issue	Medium	● Resolved
LAF-08	Missing Initialization Of <code>ReentrancyGuardUpgradeable</code> In <code>LayerEdgeStaking</code> Contract	Logical Issue	Minor	● Resolved

ID	Title	Category	Severity	Status
LAF-09	Check-Effect-Interaction Pattern Violated	Logical Issue	Minor	● Resolved
LAF-01	Inconsistency Between <code>getCurrentTier</code> Function Logic And <code>stakerTierHistory</code> Usage	Design Issue	Informational	● Acknowledged

LAF-02 | CONTRACT UPGRADE CENTRALIZATION RISK

Category	Severity	Location	Status
Centralization	● Centralization	stake/LayerEdgeStaking.sol (05/07-6c78eb): 20, 21, 22, 23	● Acknowledged

Description

In the contract `LayerEdgeStaking`, the role `owner` has the authority to update the implementation contract behind the `LayerEdgeStaking` contract.

Any compromise to the owner account may allow a hacker to take advantage of this authority and change the implementation contract which is pointed by proxy and therefore execute potential malicious functionality in the implementation contract.

Recommendation

We recommend that the team make efforts to restrict access to the admin of the proxy contract. A strategy of combining a time-lock and a multi-signature ($\frac{2}{3}$, $\frac{3}{5}$) wallet can be used to prevent a single point of failure due to a private key compromise. In addition, the team should be transparent and notify the community in advance whenever they plan to migrate to a new implementation contract.

Here are some feasible short-term and long-term suggestions that would mitigate the potential risk to a different level and suggestions that would permanently fully resolve the risk.

Short Term:

A combination of a time-lock and a multi signature ($\frac{2}{3}$, $\frac{3}{5}$) wallet mitigate the risk by delaying the sensitive operation and avoiding a single point of key management failure.

- A time-lock with reasonable latency, such as 48 hours, for awareness of privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to a private key compromised;
AND
- A medium/blog link for sharing the time-lock contract and multi-signers addresses information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.

- Provide a link to the **medium/blog** with all of the above information included.

Long Term:

A combination of a time-lock on the contract upgrade operation and a DAO for controlling the upgrade operation mitigate the contract upgrade risk by applying transparency and decentralization.

- A time-lock with reasonable latency, such as 48 hours, for community awareness of privileged operations;
AND
- Introduction of a DAO, governance, or voting module to increase decentralization, transparency, and user involvement;
AND
- A medium/blog link for sharing the time-lock contract, multi-signers addresses, and DAO information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

Permanent:

Renouncing ownership of the `admin` account or removing the upgrade functionality can *fully* resolve the risk.

- Renounce the ownership and never claim back the privileged role;
OR
- Remove the risky functionality.

Note: we recommend the project team consider the long-term solution or the permanent solution. The project team shall make a decision based on the current state of their project, timeline, and project resources.

I Alleviation

[LayerEdge, 05/13/2025]:

The team acknowledged the issue and decided to remain unchanged in the scope of the audit.

[CertiK, 05/20/2025]:

It is suggested to implement the aforementioned methods to avoid centralized failure. Also, it strongly encourages the project team to periodically revisit the private key security management of all addresses related to centralized roles.

LAF-03 | CENTRALIZATION RELATED RISKS

Category	Severity	Location	Status
Centralization	● Centralization	stake/LayerEdgeStaking.sol (05/07-6c78eb): 232, 256, 294, 305, 310, 318, 326, 801	● Acknowledged

Description

In the contract `LayerEdgeStaking`, the role `_owner` has authority over the functions shown below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and authorize contract upgrades for new implementations, withdraw rewards amount to themselves, set the compounding status, update the APY rate for a specified tier, pause contract operations, update all APYs and their history, unpause the contract, and set the minimum stake amount.

- `updateTierAPY()`
- `updateAllAPYs()`
- `withdrawRewards()`
- `pause()`
- `unPause()`
- `setMinStakeAmount()`
- `setCompoundingStatus()`
- `_authorizeUpgrade()`

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets.

Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (2/3, 3/5) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;

AND

- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;\

AND

- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;\

AND

- Introduction of a DAO/governance/voting module to increase transparency and user involvement.\

AND

- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.\

OR

- Remove the risky functionality.

I Alleviation

[LayerEdge, 05/13/2025]:

The team acknowledged the issue and decided to remain unchanged in the scope of the audit.

[CertiK, 05/20/2025]:

It is suggested to implement the aforementioned methods to avoid centralized failure. Also, it strongly encourages the project team to periodically revisit the private key security management of all addresses related to centralized roles.

LAF-04 | INCORRECT REMOVAL FROM FENWICK TREE CAUSING ARITHMETIC OVERFLOW/UNDERFLOW

Category	Severity	Location	Status
Logical Issue	Major	library/FenwickTree.sol (05/07-6c78eb): 13; stake/LayerEdgeStaking.sol (05/07-6c78eb): 674	Resolved

Description

The `unstake()` and `unstakeNative()` functions in the smart contract allow stakers to unstake specific amounts of tokens multiple times. Each unstaking operation removes the staker from the `stakerTree`. However, the `stakerTree` is updated improperly upon multiple unstaking actions, leading to potential overflow/underflow errors due to improper type conversion in the `update()` function.

The main issue arises in the `_unstake` function where the tree is updated to remove the user:

```
function _unstake(uint256 amount, address userAddr, bool isNative) internal {
    ...
    // Update tree to remove user
    @> stakerTree.update(user.joinId, -1);
    ...
}
```

When a staker unstakes a second time, the `update()` function improperly convert `int256` to `uint256` within the `update()` function leads to potential overflow:

```
function update(Tree storage self, uint256 index, int256 delta) internal {
    ...
    while (index <= self.size) {
    @> self.data[index] = uint256(int256(self.data[index]) + delta);
        index += lsb(index);
    }
}
```

`int256(self.data[index]) + delta` results in `-1`, converting this to `uint256` will set it to the maximum possible value of a `uint256`, which can later cause overflow or underflow errors during tree queries(added to the `sum`):

```
function query(Tree storage self, uint256 index) internal view returns (uint256 sum)
{
    while (index > 0) {
    @>      sum += self.data[index];
            index -= lsb(index);
    }
}
```

■ Proof of Concept

The following test shows the issue mentioned above.


```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.20;

import {Test, console2} from "forge-std/Test.sol";
import "../src/LayerEdgeStaking.sol";
import "../src/@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract LayerEdgeTest is Test {

    address private Bob = address(0x123);
    address private Alice = address(0x456);
    address private Paul = address(0x455);
    address private Admin = address(0x111);

    address wbnb = 0xbb4CdB9CBd36B01bD1cBaEBF2De08d9173bc095c;

    LayerEdgeStaking stake;
    LayerEdgeToken token;

    FenwickTree.Tree public stakerTree;

    using FenwickTree for FenwickTree.Tree;

    function setUp() public {
        // token = new LayerEdgeToken("Layer Edge Token", "LE", 1000000000 * 1e18,
Admin);
        stake = new LayerEdgeStaking();
        stake.initialize(wbnb, Admin);
    }

    function testStaking() public {
        deal(wbnb, Bob, 1000);
        vm.warp(1000);
        vm.startPrank(Bob);
        IWETH(wbnb).approve(address(stake), ~uint256(0));
        stake.stake(1000);
        vm.stopPrank();

        deal(wbnb, Alice, 1000);
        vm.startPrank(Alice);
        IWETH(wbnb).approve(address(stake), ~uint256(0));
        stake.stake(1000);
        vm.stopPrank();

        deal(wbnb, Paul, 1000);
        vm.startPrank(Paul);
        IWETH(wbnb).approve(address(stake), ~uint256(0));
        stake.stake(1000);
        vm.stopPrank();
    }
}
```

```
    vm.warp(1000 + 7 days);
    vm.startPrank(Bob);
    // stake.claimInterest();
    stake.unstake(500);
    stake.unstake(200);
    stake.unstake(300);
    // stake.unstake(100);
    // stake.unstake(100);
    // stake.unstake(100);
    vm.stopPrank();

    deal(wbnb, Paul, 1000);
    vm.startPrank(Paul);
    IWETH(wbnb).approve(address(stake), ~uint256(0));
    stake.stake(1000);
    vm.stopPrank();

}

}
```

Logs:

[illegible]

[illegible]

```

    └─ emit SlotFound(who: 0xbb4CdB9CBd36B01bD1cBaEBF2De08d9173bc095c, fsig:
0x70a08231, keyHash:
0xc6444dececf3df00ebee2dbc69d916fb306f9df6c62bb68d0575a5b2073babba, slot:
22867195542817545563399774562045346535038657056791885398317771824687611339981
[2.286e76])
    └─ [534]
0xbb4CdB9CBd36B01bD1cBaEBF2De08d9173bc095c::balanceOf(0x00000000000000000000000000000000
000000000455) [staticcall]
    |   └─ 0x0000000000000000000000000000000000000000000000000000000000000000
    |   └─ [0] VM::load(0xbb4CdB9CBd36B01bD1cBaEBF2De08d9173bc095c,
0x328e5fbf905c4af27bd38c385169e8ecac654305996451b615f180039e8764cd) [staticcall]
    |   └─ 0x0000000000000000000000000000000000000000000000000000000000000000
    |   └─ [0] VM::store(0xbb4CdB9CBd36B01bD1cBaEBF2De08d9173bc095c,
0x328e5fbf905c4af27bd38c385169e8ecac654305996451b615f180039e8764cd,
0x000000000000000000000000000000000000000000000000000000000000003e8)
    |   └─ ()
    |   └─ [0] VM::startPrank(0x0000000000000000000000000000000000000000000000455)
    |   └─ ()
    |   └─ [24420] 0xbb4CdB9CBd36B01bD1cBaEBF2De08d9173bc095c::approve(LayerEdgeStaking:
[0x5615dEB798BB3E4dFa0139dFa1b3D433Cc23b72f],
115792089237316195423570985008687907853269984665640564039457584007913129639935
[1.157e77])
    |   └─ emit Approval(owner: 0x000000000000000000000000000000000000000000000455, spender:
LayerEdgeStaking: [0x5615dEB798BB3E4dFa0139dFa1b3D433Cc23b72f], value:
115792089237316195423570985008687907853269984665640564039457584007913129639935
[1.157e77])
    |   └─ 0x0000000000000000000000000000000000000000000000000000000000000001
    |   └─ [265240] LayerEdgeStaking::stake(1000)
    |   └─ [2740]
0xbb4CdB9CBd36B01bD1cBaEBF2De08d9173bc095c::transferFrom(0x00000000000000000000000000000000
000000000000455, LayerEdgeStaking: [0x5615dEB798BB3E4dFa0139dFa1b3D433Cc23b72f],
1000)
    |   |   └─ emit Transfer(from: 0x000000000000000000000000000000000000000000000455, to:
LayerEdgeStaking: [0x5615dEB798BB3E4dFa0139dFa1b3D433Cc23b72f], value: 1000)
    |   |   └─ 0x0000000000000000000000000000000000000000000000000000000000000001
    |   |   └─ [0] console::log(stake user.joinId:, 3) [staticcall]
    |   |   └─ ()
    |   |   └─ [0] console::log(stake query:, 3) [staticcall]
    |   |   └─ ()
    |   |   └─ emit Staked(user: 0x00000000000000000000000000000000000000000000455, amount:
1000, tier: 3)
    |   |   └─ ()
    |   |   └─ [0] VM::stopPrank()
    |   |   └─ ()
    |   |   └─ [0] VM::warp(605800 [6.058e5])
    |   |   └─ ()
    |   |   └─ [0] VM::startPrank(0x00000000000000000000000000000000000000000000123)
    |   |   └─ ()
    |   |   └─ [113889] LayerEdgeStaking::unstake(500)
    |   |   └─ [0] console::log(unstake user.joinId:, 1) [staticcall]
    |   |   └─ ()

```

[illegible]

```

|   └─ ← 0x0000000000000000000000000000000000000000000000000000000000000000
|   └─ [0] VM::store(0xbb4CdB9CBd36B01bD1cBaEBF2De08d9173bc095c,
0x328e5fbf905c4af27bd38c385169e8ecac654305996451b615f180039e8764cd,
0x000000000000000000000000000000000000000000000000000000000000003e8)
|   └─ ← ()
|   └─ [0] VM::startPrank(0x00000000000000000000000000000000000000000000000000000000000000455)
|   └─ ← ()
|   └─ [2420] 0xbb4CdB9CBd36B01bD1cBaEBF2De08d9173bc095c::approve(LayerEdgeStaking:
[0x5615dEB798BB3E4dFa0139dFa1b3D433Cc23b72f],
115792089237316195423570985008687907853269984665640564039457584007913129639935
[1.157e77])
|   └─ emit Approval(owner: 0x00000000000000000000000000000000000000000000000000000000000000455, spender:
LayerEdgeStaking: [0x5615dEB798BB3E4dFa0139dFa1b3D433Cc23b72f], value:
115792089237316195423570985008687907853269984665640564039457584007913129639935
[1.157e77])
|   └─ ← 0x000000000000000000000000000000000000000000000000000000000000000001
|   └─ [32726] LayerEdgeStaking::stake(1000)
|   └─ [2740]
0xbb4CdB9CBd36B01bD1cBaEBF2De08d9173bc095c::transferFrom(0x0000000000000000000000000000000000000000000000000000000000000000455, LayerEdgeStaking: [0x5615dEB798BB3E4dFa0139dFa1b3D433Cc23b72f],
1000)
|   └─ emit Transfer(from: 0x00000000000000000000000000000000000000000000000000000000000000455, to:
LayerEdgeStaking: [0x5615dEB798BB3E4dFa0139dFa1b3D433Cc23b72f], value: 1000)
|   └─ ← 0x000000000000000000000000000000000000000000000000000000000000000001
|   └─ ← "Arithmetic over/underflow"
└─ ← "Arithmetic over/underflow"

```

Test result: FAILED. 0 passed; 1 failed; 0 skipped; finished in 4.54s
Ran 1 test suites: 0 tests passed, 1 failed, 0 skipped (1 total tests)

Failing tests:

Encountered 1 failing test in test/LayerEdge.t.sol:LayerEdgeTest
[FAIL. Reason: Arithmetic over/underflow] testStaking() (gas: 2612799)

Encountered a total of 1 failing tests, 0 tests succeeded

Recommendation

Recommend relocating the removal logic to the "fully unstake" block to ensure that stakers are removed from the staker tree only once.

Alleviation

[LayerEdge, 05/13/2025]:

The team heeded the advice and resolved the issue in the commit [2cefef545d6aa628b851d6851c721b623ddb680f](#).

LAF-06 INCORRECT TIER PROMOTION DUE TO MISSING SUFFICIENT CONDITION CHECKS IN `_checkBoundariesAndRecord` FUNCTION

Category	Severity	Location	Status
Logical Issue	● Major	stake/LayerEdgeStaking.sol (05/07-6c78eb): 777~780, 785~789	● Resolved

Description

The `_checkBoundariesAndRecord()` function recalculates and updates the tier of a staker at a boundary rank whenever the `activeStakerCount` in the contract changes. The `findByCumulativeFrequency()` function identifies the affected index based on the updated `crossRank`, the tier of the user corresponding to `joinIdCross` is updated. However, because the `_checkBoundariesAndRecord()` function does not validate whether the index and user actually meet the tier promotion requirements (such as the staking status of user), it may incorrectly promote an ineligible staker to a higher tier.

```
function _checkBoundariesAndRecord(bool isRemoval) internal {
    // recompute thresholds
    ...

    // for each boundary, if it shifted by ±1, find the user crossing
    if (new_t1 != old_t1) {
        // someone moved across Tier1-Tier2
        // the user at rank = min(old_t1, new_t1)+1 if promotion, or old_t1 if
demotion
        uint256 crossRank = new_t1 > old_t1
            ? new_t1 // promotion: the one newly entering Tier1
            : old_t1; // demotion: the one kicked out of Tier1
        uint256 joinIdCross = stakerTree.findByCumulativeFrequency(crossRank);
    @> address userCross = stakerAddress[joinIdCross];
        Tier toTier = _computeTierByRank(joinIdCross, n);
        _recordTierChange(userCross, toTier);
    }

    if (new_t2 != old_t2) {
        // Tier2-Tier3 boundary
        uint256 crossRank = new_t2 > old_t2 ? new_t2 : old_t2;
        uint256 joinIdCross = stakerTree.findByCumulativeFrequency(crossRank);
    @> address userCross = stakerAddress[joinIdCross];
        Tier toTier = _computeTierByRank(joinIdCross, n);
        _recordTierChange(userCross, toTier);
    }
}
```

In contrary, if a user was removed from the tree, it is hard to determine the cross rank because the removed user may be in the cross rank.

Scenario

In the POC, a user at `index == 2` who has only staked a small amount — below the `minStakeAmount` — should logically belong to `tier3`. But due to their low index, a change in `activeStakerCount` causes the rank at `index == 2` to be recalculated as `tier1`. As a result, the user is incorrectly promoted to `tier1` despite not meeting the qualifications.

Proof of Concept

```
function test_Incorrect_Tier_Promotion() public {
    vm.deal(Alice, 300 ether);
    vm.startPrank(Alice);
    staking.stakeNative{value: 300 ether}();

    vm.stopPrank();

    _batchStake(8, 300 ether);

    address user = users[0];          // rank == 2

    uint256 len = staking.stakerTierHistoryLength(user);
    LayerEdgeStaking.Tier tier_before_change;
    (, tier_before_change, ) = staking.stakerTierHistory(user, len-1);
    // In fact, the user's tier was also updated during the staking process.
    When stakerCount increased from 1 to 2, the number of tier2 count expanded,
    resulting in the user being immediately promoted from tier3 to tier2 upon staking.
    console.log("user's tier before promotion:", uint8(tier_before_change));

    vm.deal(Bob, 300 ether);
    vm.startPrank(Bob);
    staking.stakeNative{value: 300 ether}();

    len = staking.stakerTierHistoryLength(user);
    LayerEdgeStaking.Tier tier_after_change;
    (, tier_after_change, ) = staking.stakerTierHistory(user, len-1);
    console.log("user's tier after promotion:", uint8(tier_after_change));
}
```

```
[PASS] test_Incorrect_Tier_Promotion() (gas: 4106763)
Logs:
  user's tier before promotion: 2
  user's tier after promotion: 1

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.86ms (1.54ms CPU
time)
```

Recommendation

It is recommended to effectively verify the user's stake status, especially during promotion.

Alleviation

[LayerEdge, 05/13/2025]:

The team heeded the advice and resolved the issue in commit [2cefef545d6aa628b851d6851c721b623ddb680f](#).

We are removing the user from the system if his stake amount is less than minStake(related to LAF-04)

The tier system is based only on FCFS conditions and not dependent on the amount of stake. But the only special condition is if the user unstaked less than minStake amount he will permanently moved to Tier3 . Since the user who completely unstaked or whose balance is less than minStake is removed out of the system/tree, the checkBoundariesAndRecord() function no need to track anything related to stake amount.

LAF-05 | INEFFICIENT FUNCTION CALL IN `_checkBoundariesAndRecord`

Category	Severity	Location	Status
Logical Issue	● Medium	stake/LayerEdgeStaking.sol (05/07-6c78eb): 651, 689	● Resolved

Description

The `_checkBoundariesAndRecord()` function is designed to recompute a staker's tier whenever there is a change in the total staker count. Specifically, this function is triggered when the total staker count increases or decreases by 1, which occurs when a new staker adds tokens to the contract or when an existing staker fully withdraws their stake. The function then adjusts the staker's position between different tiers: tier1 and tier2, or tier2 and tier3. Here is a code snippet illustrating the relevant section of the function:

```
function _stake(uint256 amount, address userAddr, bool isNative) internal {
    ...
    _checkBoundariesAndRecord(false);
    ...
}

function _unstake(uint256 amount, address userAddr, bool isNative) internal {
    ...
    _checkBoundariesAndRecord(true);
    ...
}

function _checkBoundariesAndRecord(bool isRemoval) internal {
    // recompute thresholds
    uint256 n = activeStakerCount;
    uint256 oldN = isRemoval ? n + 1 : n - 1; // for removal we call after
    decrement; for add we call after increment

    // old thresholds (before this tx's change)
    (uint256 old_t1, uint256 old_t2,) = getTierCountForStakerCount(oldN);
    // new thresholds
    (uint256 new_t1, uint256 new_t2,) = getTierCountForStakerCount(n);

    // for each boundary, if it shifted by ±1, find the user crossing
    if (new_t1 != old_t1) {
        // someone moved across Tier1→Tier2
        // the user at rank = min(old_t1, new_t1)+1 if promotion, or old_t1 if
        demotion
        uint256 crossRank = new_t1 > old_t1
            ? new_t1 // promotion: the one newly entering Tier1
            : old_t1; // demotion: the one kicked out of Tier1
        uint256 joinIdCross = stakerTree.findByCumulativeFrequency(crossRank);
        address userCross = stakerAddress[joinIdCross];
        Tier toTier = _computeTierByRank(joinIdCross, n);
        _recordTierChange(userCross, toTier);
    }

    if (new_t2 != old_t2) {
        // Tier2→Tier3 boundary
        uint256 crossRank = new_t2 > old_t2 ? new_t2 : old_t2;
        uint256 joinIdCross = stakerTree.findByCumulativeFrequency(crossRank);
        address userCross = stakerAddress[joinIdCross];
        Tier toTier = _computeTierByRank(joinIdCross, n);
        _recordTierChange(userCross, toTier);
    }
}
```

However, there is a logical inefficiency within the function's invocation strategy. **If a staker stakes or unstakes multiple times without affecting the overall staker count, calling `_checkBoundariesAndRecord()` is unnecessary since the total staker count remains unchanged.**

Recommendation

To optimize the use of the `_checkBoundariesAndRecord()` function, modify its invocation conditions such that it is only called when the total staker count changes.

Alleviation

[LayerEdge, 05/13/2025]:

The team heeded the advice and resolved the issue in the commit [2cefef545d6aa628b851d6851c721b623ddb680f](#).

[CertiK, 05/14/2025]:

The LayerEdge team has added a new mapping variable called `totalStakersSnapshot` to track changes in the total number of staker accounts. However, this snapshot is only taken when a staker stakes or unstakes. If a staker stakes a second time, the total staker count may have already increased. Even if there are no changes in the total staker count at that moment, the logic still calls `_checkBoundariesAndRecord()` because the latest total staker count differs from the snapshot value associated with the staker. It is recommended to call the `_checkBoundariesAndRecord()` function only when the total staker count actually changes. For example:

```
activeStakerCount++;  
_checkBoundariesAndRecord(false);
```

```
activeStakerCount--;  
_checkBoundariesAndRecord(true);
```

[LayerEdge, 05/19/2025]:

The team heeded the advice and resolved the issue in the commit [dd7dd86477c6670a3fbdd6d0b4adc20bf47cd6d6](#).

LAF-07 | MISSING TIER RECALCULATION IN `compoundInterest()` FUNCTION FOR STAKERS

Category	Severity	Location	Status
Logical Issue, Inconsistency	● Medium	stake/LayerEdgeStaking.sol (05/07-6c78eb): 212, 221, 643~648	● Resolved

Description

In the current implementation of the smart contract, there is a potential issue with the tier recalculation for stakers using the `compoundInterest()` function. The `_stake()` function correctly checks whether a staker's balance meets or exceeds the `minStakeAmount` threshold and recomputes the staker's tier accordingly, as shown in the code snippet below:

```
if (user.balance >= minStakeAmount) {
    tier = _computeTierByRank(rank, activeStakerCount);
    user.isFirstDepositMoreThanMinStake = true;
}

_recordTierChange(userAddr, tier);
```

However, stakers have the option to increase their staked tokens through the `compoundInterest()` function. This function does not account for recalculating and recording the staker's tier if the compounded interest causes their total staked amount to meet or exceed the `minStakeAmount`. As a result, the staker's tier may not accurately reflect their total contribution, potentially affecting benefits tied to their tier status.

Recommendation

To ensure accurate tier assignment, modify the `compoundInterest()` function to include a tier recalculation mechanism similar to the one found in the `_stake()` function. This modification should evaluate whether the newly compounded total exceeds the `minStakeAmount` and update the staker's tier as needed. Here's a suggested approach:

1. After the interest is compounded, check if the total staked amount now meets or exceeds `minStakeAmount`.
2. If so, recompute the tier using `_computeTierByRank()` and record the tier change accordingly.

Alleviation

[LayerEdge, 05/13/2025]:

The team heeded the advice and resolved the issue in the commit [2cefef545d6aa628b851d6851c721b623ddb680f](#).

LAF-10 | UNSTAKE MAY FAIL DUE TO INVALID TIER

Category	Severity	Location	Status
Denial of Service	● Medium	stake/LayerEdgeStaking.sol (05/07-6c78eb): 381, 428, 445, 537, 77, 786	● Resolved

Description

It is crucial for the stake protocol to ensure users can safely retrieve their principal. During the unstaking process, interest are calculated first, and the user's tier will be judged.

```
function getTierAPYForPeriod(Tier tier, uint256 startTime, uint256 endTime)
public view returns (uint256) {
    ...
    require(tier >= Tier.Tier1 && tier <= Tier.Tier3, "Invalid tier");
    ...
}
```

If the tier is not tier1, tier2, or tier3, the entire transaction will be rolled back. The user's tier might be computed through the `_computeTierByRank()` function that could result in `Tier.None`. If a user is mistakenly set to `Tier.None`, they will be unable to withdraw their principal.

```
function _computeTierByRank(uint256 rank, uint256 totalStakers) internal pure
returns (Tier) {
    if (rank == 0 || rank > totalStakers) return Tier.None;
    ...
}
```

Recommendation

To ensure users can withdraw their principal, the protocol should handle the case of `Tier.None` more gracefully. Consider implementing a default behavior or provision that allows users to proceed with withdrawal if they are assigned as `Tier.None`.

Alleviation

[LayerEdge, 05/13/2025]:

The team heeded the advice and resolved the issue in the commit [2cefef545d6aa628b851d6851c721b623ddb680f](#).

[Certik, 05/14/2025]:

To guarantee successful unstaking, it is crucial to validate whether the `joinIdCross` retrieved via the

`findByCumulativeFrequency()` function is valid. An invalid `joinIdCross` may lead to incorrect tier calculations.

[LayerEdge, 05/19/2025]:

The team heeded the advice and resolved the issue in the commit [dd7dd86477c6670a3fbd6d0b4adc20bf47cd6d6](#).

LAS-01 | INCORRECT TIER UPDATES DUE TO RANK MISMATCH

Category	Severity	Location	Status
Logical Issue	● Medium	src/stake/LayerEdgeStaking.sol (05/13-2cefef5): 790~792	● Acknowledged

Description

In the `_checkBoundariesAndRecord()` function, `tier` promotions and demotions for certain `joinIds` are recalculated based on the current `activeStakerCount`. However, since the function relies on the **counts** of each tier rather than the **starting index** of each tier, it may incorrectly determine which `joinId` should be updated. Specifically, when the count of `Tier2` changes, the computed results in `getTierCountForStakerCount()` function do not necessarily reflect the actual ranks where promotions or demotions occur. This can cause the tier of users who were truly promoted or demoted to remain outdated.

```
function _checkBoundariesAndRecord(bool isRemoval) internal {
    ...

    // for each boundary, if it shifted by ±1, find the user crossing
    ...

    if (new_t2 != old_t2) {
        // Tier2→Tier3 boundary
        @> uint256 crossRank = new_t2 > old_t2 ? new_t2 : old_t2;
        uint256 joinIdCross = stakerTree.findByCumulativeFrequency(crossRank);
        address userCross = stakerAddress[joinIdCross];
        Tier toTier = _computeTierByRank(joinIdCross, n);
        _recordTierChange(userCross, toTier);
    }
}
```

For example, suppose there are currently 6 `activeStakers`: 1 in Tier1, 1 in Tier2, and 4 in Tier3. When a new user joins, the `getTierCountForStakerCount()` function calculates the new tier distribution as `new_t1` = 1 and `new_t2` = 2. As a result, the function will update the tier of the user with `crossRank` = 2. However, the actual promotion should have applied to the user with `crossRank` = 3. This mismatch leads to incorrect tier updates.

Conversely, when a user is removed from the tree, the total number of staker accounts decreases by 1. This triggers a tier recalculation via the `getTierCountForStakerCount()` function, but the cross-tier impact cannot be reliably predicted due to the randomness of user removal. For example, suppose there are currently 7 `activeStakers`: 1 in Tier1, 2 in Tier2, and 4 in Tier3. After remove one user, there will be 1 Tier1, 1 Tier2 and 4 Tier3. Let consider two cases:

1. If the user in Tier1 was removed, then the first user in Tier2 should be moved to Tier1.
2. If the first user in Tier2 was removed, then no users need to be moved.

However, the current logic moved the second user in Tier2 to Tier3 no matter which user has been removed.

Recommendation

It is recommended to update the promotion/demotion logic to identify the actual users whose ranks are affected by changes in `activeStakerCount`.

Alleviation

[LayerEdge, 05/19/2025]:

The team acknowledged the issue and updated the codebase in the commit [dd7dd86477c6670a3fbdd6d0b4adc20bf47cd6d6](#).

[CertiK, 05/19/2025]:

The `getTierCountForStakerCount()` function calculates the number of users in `Tier1` and `Tier2` based on predefined ratios. As a result, changes in the total number of stakers may cause both `Tier1` and `Tier2` counts to increase simultaneously. In such cases, some users originally in `Tier2` may be promoted to `Tier1`, which creates additional slots in `Tier2` and triggers multiple IDs' tier updates across ranks.

However, the current logic only updates tiers at the boundary indices where count changes are detected, rather than all affected user IDs. For example, if there are initially 9 stakers with tier distribution (1, 2, 6), and one more user joins, the distribution changes to (2, 3, 5). Although three users' tiers need to be updated, the logic only updates the tiers at ranks 2 and 5 — the changed boundaries — potentially leaving other affected users with outdated tier information.

In the latest commit, the team has expanded the `_checkBoundariesAndRecord()` function to account for more potential Tier changes, but several issues remain:

1. Cross-Rank Calculation in Promotions and Demotions

The `getTierCountForStakerCount()` function calculates the number of `Tier1` and `Tier2` users based on fixed proportions. This means both `Tier1` and `Tier2` counts may change at the same time, and an ID previously classified as `Tier2` might be promoted to `Tier1`, thereby freeing up more slots for `Tier2`, which means that not only the boundary value needs to update the Tier. However, the current implementation only updates the Tiers at the changed boundary indexes and does not account for all IDs that may require updates.

Example: With 9 stakers, the Tier distribution might be (1, 2, 6). After a new user joins, it becomes (2, 3, 5). While three users' Tiers should be updated (Id 2 promoted to Tier1, Ids 4 and 5 promoted to Tier2), the logic only updates the users at the new boundaries, i.e., IDs 2 and 5.

2. Redundant Logic

When users unstake, even if the `Tier1` or `Tier2` counts remain unchanged, other users' Tiers might still need to be updated. In contrast, when staking, new users are added sequentially, and if the total staker count does not increase, existing users' Tiers typically remain the same. Thus, parts of the current logic may be unnecessary in the staking scenario.

```

    if (new_t1 != 0) {
        ...
    } else if (!isRemoval) {
        // If a user was added, the user at position old_t1 might have
changed tiers
        _findAndRecordTierChange(old_t1, n);
    }
}

if (new_t1 + new_t2 > 0) {
    // Ensure there are stakers in Tier 1 or Tier 2
    ...
} else if (!isRemoval) {
    // If a user was added, the user at old boundary between Tier 2
and Tier 3 might have changed
    uint256 crossRank = old_t1 + old_t2; // Old boundary position
    _findAndRecordTierChange(crossRank, n);
}
}

```

3. Inconsistent Tier Boundary Handling

When updating `Tier2` boundaries, `crossRank` is calculated as the sum of the `Tier1` and `Tier2` counts. However, if both `Tier1` and `Tier2` counts change simultaneously, inconsistently using `old` vs `new` values in this calculation can lead to inaccurate rank determination and incorrect Tier updates.

```

if (new_t1 + new_t2 > 0) {
    // Ensure there are stakers in Tier 1 or Tier 2
    if (new_t2 != old_t2) {
        // Tier 2 boundary changed - handle as before
        uint256 crossRank;
        if (new_t2 > old_t2) {
            // Promotion
            crossRank = new_t1 + new_t2; // Add new_t1 count to land on
correct index/rank
        } else {
            // Demotion
            @> crossRank = new_t1 + old_t2; // Add new_t1 count to land on
correct index/rank
        }
        _findAndRecordTierChange(crossRank, n);
    }
    // Handle case where Tier 2 count stays the same
    ...
}

```

Overall, the `_checkBoundariesAndRecord()` function is designed to update users' Tier statuses dynamically as the number of stakers changes. However, it currently focuses on boundary values and specific edge cases, failing to account for broader impacts on all affected users in the Tree.

It is recommended that the team refactor the `_checkBoundariesAndRecord()` function to more comprehensively handle all scenarios where Tier changes may occur, ensuring the consistency and correctness of user Tier updates.

[LayerEdge, 05/20/2025]:

The team acknowledged the issue and updated the codebase in the commit [b7457074df7cdf5c5c2cdfdf1a70c7c200d27986](#).

[CertiK, 05/20/2025]:

The algorithm has undergone basic testing but may not cover all possible use cases. It is recommended to validate the algorithm with as many test cases as possible before deployment to ensure its correctness and stability in your specific environment.

LAS-02 | INCORRECT RANK ASSIGNMENT DUE TO MISUSED INPUT IN TIER COMPUTATION

Category	Severity	Location	Status
Logical Issue	● Medium	stake/LayerEdgeStaking.sol (05/07-6c78eb): 777~779, 786~788	● Resolved

Description

The `_checkBoundariesAndRecord()` function is responsible for updating a staker's tier when their rank changes as a result of fluctuations in the total number of stakers. However, the function incorrectly calculates the new tier due to a mistake in the input passed to the `_computeTierByRank()` function. Instead of using the rank of the staker, the function erroneously uses the join ID of the staker. This results in an incorrect tier being set for the staker. Below is the relevant code snippet that highlights the issue:

```
uint256 joinIdCross = stakerTree.findByCumulativeFrequency(crossRank);
address userCross = stakerAddress[joinIdCross];
Tier toTier = _computeTierByRank(joinIdCross, n);
```

Recommendation

Recommend passing the rank value as the first parameter to the `_computeTierByRank()` function.

Alleviation

[LayerEdge, 05/19/2025]:

The team heeded the advice and resolved the issue by passing correct rank value into the `_computeTierByRank()` function in commit [dd7dd86477c6670a3fbdd6d0b4adc20bf47cd6d6](#).

LAF-08 | MISSING INITIALIZATION OF `ReentrancyGuardUpgradeable` IN `LayerEdgeStaking` CONTRACT

Category	Severity	Location	Status
Logical Issue	● Minor	stake/LayerEdgeStaking.sol (05/07-6c78eb): 121	● Resolved

Description

The `LayerEdgeStaking` contract inherits from the `ReentrancyGuardUpgradeable` contract but fails to explicitly initialize it. In Solidity, when a contract extends another, it is crucial to ensure that all parent contracts' initializer functions are called appropriately. This typically involves calling the `__ReentrancyGuard_init()` function within the initializer of the derived contract to ensure the internal states of `ReentrancyGuardUpgradeable` are correctly set up. The missing initialization may lead to undefined behavior and potential vulnerabilities, especially since reentrancy protection might not be enabled as expected.

Recommendation

To fix the issue, ensure that the `__ReentrancyGuard_init()` function is called within the `initialize()` function of the `LayerEdgeStaking` contract. This guarantees that the `ReentrancyGuardUpgradeable` contract's state is properly initialized, enabling the reentrancy protection mechanism as intended.

Alleviation

[LayerEdge, 05/13/2025]:

The team heeded the advice and resolved the issue in the commit [2cefef545d6aa628b851d6851c721b623ddb680f](#).

LAF-09 | CHECK-EFFECT-INTERACTION PATTERN VIOLATED

Category	Severity	Location	Status
Logical Issue	● Minor	stake/LayerEdgeStaking.sol (05/07-6c78eb): 278~288	● Resolved

Description

In the `depositRewards()` function of the `LayerEdgeStaking` contract, an external token transfer (`stakingToken.transferFrom`) is performed before updating the rewards reserve. This violates the `Checks-Effects-Interactions` pattern and could open up vulnerabilities if the `stakingToken` token has special behaviors like callbacks.

```
function depositRewards(uint256 amount) external whenNotPaused {
    require(amount > 0, "Cannot deposit zero amount");

    // Transfer tokens from sender to contract
    require(stakingToken.transferFrom(msg.sender, address(this), amount), "Token
transfer failed");

    // Update rewards reserve
    rewardsReserve += amount;

    emit RewardsDeposited(msg.sender, amount);
}
```

Recommendation

We recommend using the `Checks-Effects-Interactions` pattern or add `nonReentrant` modifier of `ReentrancyGuard` contract for the aforementioned functions to prevent reentrancy attack.

Alleviation

[LayerEdge, 05/13/2025]:

The team heeded the advice and resolved the issue in the commit [2cefef545d6aa628b851d6851c721b623ddb680f](#).

LAF-01 | INCONSISTENCY BETWEEN `getCurrentTier` FUNCTION LOGIC AND `stakerTierHistory` USAGE

Category	Severity	Location	Status
Design Issue	● Informational	stake/LayerEdgeStaking.sol (05/07-6c78eb): 339~357	● Acknowledged

Description

The `getCurrentTier()` function correctly determines the user's current tier by fully verifying their status. However, the actual calculation and use of `stakerTierHistory` are not aligned with the results returned by the `getCurrentTier()` function. This inconsistency may reduce code readability and could introduce potential risks.

1. Tier Mismatch After Removal:

During the unstaking process, the user is removed from the `stakerTree`, the user's rank, as determined by the `query()` function, will also be updated accordingly. If the query result is 0, the `getCurrentTier()` function will return `None` as the Tier, making them ineligible for further interest. However, if the query result is non-zero, it may duplicate with the rank of another existing index. In such cases, the `getCurrentTier()` function may mistakenly return the Tier associated with that other index, leading to an incorrect Tier being assigned to the user.

2. Invalid Promotion via Rank Boundary Change:

In another scenario, the `getCurrentTier()` function returns tier3 because the user's stakeAmount does not meet the `minStakeAmount` threshold. However, due to a rank boundary shift, the user is promoted to a higher tier and can earn interest at an undeserved higher APY, despite failing to meet the actual qualification criteria. The specific impacts of this scenario is addressed separately in finding **Incorrect Tier Promotion Due to Missing Sufficient Condition Checks** in `_checkBoundariesAndRecord` function.

Recommendation

We recommend that the team thoroughly review and align the logic across all tier-related operations, ensuring proper validation to avoid unintended behavior or reward abuse.

Alleviation

[LayerEdge, 05/13/2025]:

Issue Acknowledged.

Firstly, the tier system is only applicable based on FCFS. It doesn't depend on the amount that the user stakes. Although with only one exception, when the user's deposited amount is less than `minStakeAmount`, we are setting him to tier 3.

Any user whose tier is set to 3, because of amount less than minStake:

- They will be in tier-3 forever.
- They cannot compound and go up in tier.
- They cannot stake again.
- They will be considered out of the system.

APPENDIX | LAYEREDGE STAKING

Finding Categories

Categories	Description
Denial of Service	Denial of Service findings indicate that an attacker may prevent the program from operating correctly or responding to legitimate requests.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.

Checksum Calculation Method

The "Checksum" field in the "Audit Scope" section is calculated as the SHA-256 (Secure Hash Algorithm 2 with digest size of 256 bits) digest of the content of each file hosted in the listed source repository under the specified commit.

The result is hexadecimal encoded and is the same as the output of the Linux "sha256sum" command against the target file.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevating Your Entire **Web3** Journey

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is a leading blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.

